

Министерство Образования Республики Беларусь
УО Брестский Государственный Технический Университет
Кафедра ИИТ

Отчет по лабораторной работе № 5
По дисциплине: «Современный платформы программирования»
Специальность: «Программная инженерия»

Выполнил:
О.А. Козюта
Студент группы ПО-13
Проверил:
А.Д. Кулик
Преп.-стажёр кафедры ИИТ

Брест 2026

Цель работы: приобрести практические навыки разработки API и баз данных

Требования к оформлению отчета

Отчет по лабораторной работе должен содержать следующие разделы (примеры оформления отчетов можно найти в папке с заданиями):

- 1) Изложение цели работы.
- 2) Задание по лабораторной работе с описанием своего варианта.
- 3) Спецификации ввода-вывода программы.
- 4) Текст программы (кратко).
- 5) Выводы по проделанной работе.

Общее задание

1. Реализовать базу данных из не менее 5 таблиц на заданную тематику.

При реализации продумать типизацию полей и внешние ключи в таблицах;

2. Визуализировать разработанную БД с помощью схемы, на которой отображены все таблицы и связи между ними (пример, схема на рис. 1);

3. На языке Python с использованием SQLAlchemy реализовать подключение к БД;

4. Реализовать основные операции с данными (выборку, добавление, удаление, модификацию);

5. Для каждой реализованной операции с использованием FastAPI реализовать отдельный эндпоинт;

Базу данные можно реализовать в любой СУБД (MySQL, PostgreSQL, SQLite и др.)

- 8) База данных Прокат товаров и услуг

Код:

```
"""FastAPI application with CRUD endpoints."""
```

```
from typing import List
```

```
from fastapi import FastAPI, HTTPException, Depends
```

```
from sqlalchemy.orm import Session
```

```
from database import engine, get_db
```

```
import models
```

```
import schemas
```

```
# Create tables
```

```
models.Base.metadata.create_all(bind=engine)
```

```
app = FastAPI(title="Rental Service API", description="API for renting goods and services")
```

```
# ----- Category endpoints -----
```

```
@app.post("/categories/", response_model=schemas.CategoryResponse)
```

```
def create_category(category: schemas.CategoryCreate, db: Session = Depends(get_db)):
```

```
    db_category = models.Category(**category.dict())
```

```
    db.add(db_category)
```

```
    db.commit()
```

```
    db.refresh(db_category)
```

```
    return db_category
```

```
@app.get("/categories/", response_model=List[schemas.CategoryResponse])
def read_categories(skip: int = 0, limit: int = 100, db: Session = Depends(get_db)):
    categories = db.query(models.Category).offset(skip).limit(limit).all()
    return categories
```

```
@app.put("/categories/{category_id}", response_model=schemas.CategoryResponse)
def update_category(category_id: int, category: schemas.CategoryCreate, db: Session =
Depends(get_db)):
    db_category = db.query(models.Category).filter(models.Category.id == category_id).first()
    if not db_category:
        raise HTTPException(status_code=404, detail="Category not found")
    for key, value in category.dict().items():
        setattr(db_category, key, value)
    db.commit()
    db.refresh(db_category)
    return db_category
```

```
@app.delete("/categories/{category_id}")
def delete_category(category_id: int, db: Session = Depends(get_db)):
    db_category = db.query(models.Category).filter(models.Category.id == category_id).first()
    if not db_category:
        raise HTTPException(status_code=404, detail="Category not found")
    db.delete(db_category)
    db.commit()
    return {"ok": True}
```

----- Item endpoints -----

```
@app.post("/items/", response_model=schemas.ItemResponse)
def create_item(item: schemas.ItemCreate, db: Session = Depends(get_db)):
    db_item = models.Item(**item.dict())
    db.add(db_item)
    db.commit()
    db.refresh(db_item)
    return db_item
```

```
@app.get("/items/", response_model=List[schemas.ItemResponse])
def read_items(skip: int = 0, limit: int = 100, db: Session = Depends(get_db)):
    items = db.query(models.Item).offset(skip).limit(limit).all()
    return items
```

```
@app.put("/items/{item_id}", response_model=schemas.ItemResponse)
def update_item(item_id: int, item: schemas.ItemCreate, db: Session = Depends(get_db)):
    db_item = db.query(models.Item).filter(models.Item.id == item_id).first()
```

```

if not db_item:
    raise HTTPException(status_code=404, detail="Item not found")
for key, value in item.dict().items():
    setattr(db_item, key, value)
db.commit()
db.refresh(db_item)
return db_item

```

```

@app.delete("/items/{item_id}")
def delete_item(item_id: int, db: Session = Depends(get_db)):
    db_item = db.query(models.Item).filter(models.Item.id == item_id).first()
    if not db_item:
        raise HTTPException(status_code=404, detail="Item not found")
    db.delete(db_item)
    db.commit()
    return {"ok": True}

```

----- Customer endpoints -----

```

@app.post("/customers/", response_model=schemas.CustomerResponse)
def create_customer(customer: schemas.CustomerCreate, db: Session = Depends(get_db)):
    db_customer = models.Customer(**customer.dict())
    db.add(db_customer)
    db.commit()
    db.refresh(db_customer)
    return db_customer

```

```

@app.get("/customers/", response_model=List[schemas.CustomerResponse])
def read_customers(skip: int = 0, limit: int = 100, db: Session = Depends(get_db)):
    customers = db.query(models.Customer).offset(skip).limit(limit).all()
    return customers

```

```

@app.put("/customers/{customer_id}", response_model=schemas.CustomerResponse)
def update_customer(customer_id: int, customer: schemas.CustomerCreate, db: Session = Depends(get_db)):
    db_customer = db.query(models.Customer).filter(models.Customer.id == customer_id).first()
    if not db_customer:
        raise HTTPException(status_code=404, detail="Customer not found")
    for key, value in customer.dict().items():
        setattr(db_customer, key, value)
    db.commit()
    db.refresh(db_customer)
    return db_customer

```

```

@app.delete("/customers/{customer_id}")
def delete_customer(customer_id: int, db: Session = Depends(get_db)):

```

```

db_customer = db.query(models.Customer).filter(models.Customer.id == customer_id).first()
if not db_customer:
    raise HTTPException(status_code=404, detail="Customer not found")
db.delete(db_customer)
db.commit()
return {"ok": True}

# ----- Staff endpoints -----
@app.post("/staff/", response_model=schemas.StaffResponse)
def create_staff(staff: schemas.StaffCreate, db: Session = Depends(get_db)):
    db_staff = models.Staff(**staff.dict())
    db.add(db_staff)
    db.commit()
    db.refresh(db_staff)
    return db_staff

@app.get("/staff/", response_model=List[schemas.StaffResponse])
def read_staff(skip: int = 0, limit: int = 100, db: Session = Depends(get_db)):
    staff_list = db.query(models.Staff).offset(skip).limit(limit).all()
    return staff_list

@app.put("/staff/{staff_id}", response_model=schemas.StaffResponse)
def update_staff(staff_id: int, staff: schemas.StaffCreate, db: Session = Depends(get_db)):
    db_staff = db.query(models.Staff).filter(models.Staff.id == staff_id).first()
    if not db_staff:
        raise HTTPException(status_code=404, detail="Staff not found")
    for key, value in staff.dict().items():
        setattr(db_staff, key, value)
    db.commit()
    db.refresh(db_staff)
    return db_staff

@app.delete("/staff/{staff_id}")
def delete_staff(staff_id: int, db: Session = Depends(get_db)):
    db_staff = db.query(models.Staff).filter(models.Staff.id == staff_id).first()
    if not db_staff:
        raise HTTPException(status_code=404, detail="Staff not found")
    db.delete(db_staff)
    db.commit()
    return {"ok": True}

# ----- Rental endpoints -----
@app.post("/rentals/", response_model=schemas.RentalResponse)
def create_rental(rental: schemas.RentalCreate, db: Session = Depends(get_db)):
    # Check item availability

```

```

item = db.query(models.Item).filter(models.Item.id == rental.item_id).first()
if not item or item.quantity < 1:
    raise HTTPException(status_code=400, detail="Item not available")
db_rental = models.Rental(**rental.dict())
db.add(db_rental)
item.quantity -= 1
db.commit()
db.refresh(db_rental)
return db_rental

```

```

@app.get("/rentals/", response_model=List[schemas.RentalResponse])
def read_rentals(skip: int = 0, limit: int = 100, db: Session = Depends(get_db)):
    rentals = db.query(models.Rental).offset(skip).limit(limit).all()
    return rentals

```

```

@app.put("/rentals/{rental_id}", response_model=schemas.RentalResponse)
def update_rental(rental_id: int, rental: schemas.RentalCreate, db: Session =
Depends(get_db)):
    db_rental = db.query(models.Rental).filter(models.Rental.id == rental_id).first()
    if not db_rental:
        raise HTTPException(status_code=404, detail="Rental not found")
    for key, value in rental.dict().items():
        setattr(db_rental, key, value)
    db.commit()
    db.refresh(db_rental)
    return db_rental

```

```

@app.delete("/rentals/{rental_id}")
def delete_rental(rental_id: int, db: Session = Depends(get_db)):
    db_rental = db.query(models.Rental).filter(models.Rental.id == rental_id).first()
    if not db_rental:
        raise HTTPException(status_code=404, detail="Rental not found")
    # Return item quantity
    item = db.query(models.Item).filter(models.Item.id == db_rental.item_id).first()
    if item:
        item.quantity += 1
    db.delete(db_rental)
    db.commit()
    return {"ok": True}

```

```

@app.post("/rentals/{rental_id}/complete")
def complete_rental(rental_id: int, db: Session = Depends(get_db)):
    db_rental = db.query(models.Rental).filter(models.Rental.id == rental_id).first()
    if not db_rental:
        raise HTTPException(status_code=404, detail="Rental not found")
    if db_rental.status == "completed":

```

```
        raise HTTPException(status_code=400, detail="Already completed")
    db_rental.status = "completed"
    item = db.query(models.Item).filter(models.Item.id == db_rental.item_id).first()
    if item:
        item.quantity += 1
    db.commit()
    return {"ok": True}
```

```
@app.get("/customers/{customer_id}/rentals",
response_model=List[schemas.RentalResponse])
def customer_rentals(customer_id: int, db: Session = Depends(get_db)):
    rentals = db.query(models.Rental).filter(models.Rental.customer_id == customer_id).all()
    return rentals
```

```
# ----- Run with: uvicorn main:app --reload -----
```

```
if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="127.0.0.1", port=8000)
```

Вывод: приобретены практические навыки разработки API и баз данных